UNIVERSITAT ROVIRA i VIRGILI



SDD 2 – Specs

TAP

Recap: SDD 1

- **Steering files**: stable context about the project
  - Coverage, specificity, consistency and real usefulness
- **Specs**: one concrete task, with its acceptance criteria
  - The agent receives both: the steering and the spec
- **Today**
  - How to write a spec and check that it is good
  - How it changes with the type of task
  - How to review AI-written specs, and what they cost in tokens

Anatomy of a Good Spec

- **Context**: why the task exists
- **Goal**: what the system must do
- **Acceptance criteria**: how we check that it is right
- **Edge cases**: unusual inputs and failures
- **Out of scope**: what is not done now
- Example (illustrative):

```
# Search books by author
Context: customers cannot find books by author
Goal: GET /books?author= filters by author
Acceptance: existing author → only their books
Edge cases: unknown author, empty parameter
Out of scope: pagination, sorting
```

Spec Quality Criteria

- **Clarity**: says exactly what to do, without ambiguity
  - Would two people implement it the same way?
- **Completeness**: covers the relevant cases, including limits and errors
  - What happens if it fails or arrives empty?
- **Scoping**: asks for no more than needed, and leaves nothing important out
  - Is anything extra? Is anything missing?
- **Consistency with the steering**: respects existing conventions
  - Does it contradict any rule of the steering?

Acceptance Criteria

- A criterion is good when anyone can say, **without discussion**, whether it is met
- **Vague**
  - The search should work well
  - It must be fast and handle errors
- **Testable**
  - With the author "Cervantes", the response contains only his books
  - With an author who has no books, the response is an empty list
- **Verifiable, concrete, unambiguous**

From Criterion to Check

- **Criterion:** with an unknown author, the list is empty
- **The same criterion as a test:**

```
@Test
void unknownAuthorReturnsEmptyList() {
    var result = repo.findByAuthor("Nobody");
    assertTrue(result.isEmpty());
}
```

- No automatic test possible?
  - Write manual steps and the expected result in the spec
- Similar tests already exist?
  - The spec can point to them instead of reinventing the check

Exercise: Make Them Testable

- Rewrite each criterion so it can be checked with a test or with concrete steps
  - The search must be fast
  - It must handle errors well
  - The screen must look right
- Individual exercise; then we compare a few answers
- **Hint:** look for a number, a concrete input, a concrete output or a visible state

Four Types of Task

- **Bug fix**: what fails and what should happen
- **New feature**: what is added and where it fits
- **Refactoring**: what improves without changing behaviour
- **Performance**: how much it improves and how it is measured

- Same basic structure, but **different things to make explicit**
  - Next: the bug fix in depth, and the other three by contrast

Bug Fix: Reproduce First

- Example spec (illustrative):

```
# Bug: author filter is case sensitive
Reproduce: GET /books?author=cervantes
Actual: returns []
Expected: same books as author=Cervantes
No regression: unknown author still returns []
Out of scope: accents, partial matches
```

- **Reproduction:** exact steps to see the failure
- **Actual vs expected:** two short, observable facts
- **No regression:** what works today must stay the same
- **Out of scope:** avoids fixes nobody asked for

New Feature: Scope and Fit

- **Scope:** what is added and what stays out
  - Search by author in the API; no pagination or sorting
- **Acceptance criteria:** how we know it is finished
  - Existing author: only their books; unknown author: empty list
- **Integration:** which existing pieces are reused
  - Reuse the existing service and follow the response codes in the steering
- There is no behaviour to reproduce: the risk is **asking too much or too little**

Refactoring: Same Behaviour

- **Before:** one 120-line method that validates, calculates and saves
- **After:** three small methods, one responsibility each
- **Invariant:** the same inputs produce the same outputs
- **Verification:** the existing tests pass without being modified
- **Measurable improvement:** what changes and how it is seen
  - For example, shorter methods
- If the spec does not say what stays, the agent may **change behaviour unnoticed**

Performance: Measure First

- **Starting metric:** where we are now (example: 1200 ms)
- **Target metric:** where we want to be (example: 300 ms)
- **How it is measured**
  - Same data, same machine, several runs
  - A single timing is noise: prefer a stable metric, such as the number of database queries
- Without numbers, "faster" **cannot be verified**

Spec Templates

- **Bug fix:** context · reproduction · actual / expected · no regression · out of scope
- **New feature:** context · goal · acceptance criteria · edge cases · integration · out of scope
- **Refactoring:** context · improvement · invariant · verification · out of scope
- **Performance:** context · starting metric · target metric · how it is measured · out of scope

- All start with **context** and end with **out of scope**; what is in between depends on the type

Reviewing AI-Written Specs

- AI writes the draft; **whoever signs the spec is responsible** for it
- **Omissions**
  - Are edge cases, errors or acceptance criteria missing?
- **Ambiguities**
  - Words like "correct" or "fast" with no measure?
- **Over-specification**
  - Does it say how to code instead of what to achieve? Does it repeat the steering?
- A longer text looks more complete, but **costs more tokens**

Specs and Tokens

- A **token** is the unit a model reads and writes; it is what gets billed
- **Vague or inflated spec**
  - The agent explores and guesses → more iterations → **more tokens**
- **Clear and scoped spec**
  - The agent goes straight to the point → **fewer tokens**
- The steering files also cost tokens: they are loaded with **every task**

SDD in Professional Teams

- **Teamwork**
  - The spec is the point of agreement between who writes, implements and reviews
- **Code review of specs**
  - Fixing a paragraph is cheaper than fixing a module
- **Keeping steering files up to date**
  - An outdated steering teaches the agent habits that no longer exist
- Specs and steering files are **versioned and maintained like code**

Limits of the Method

- **SDD does not pay off for**
  - Trivial, one-line changes
  - Throwaway prototypes
  - Requirements that are still unknown
- **A lighter approach is enough**
  - A short spec of a few lines and a minimal steering with the essentials
  - Iterate, then formalise what works
- The effort of specifying should be **proportional to the risk** of the task

Questions and Discussion

- In which kind of task would you **not** write a spec?
- How would you keep a steering file up to date in a team?
- What would you ask of whoever reviews your spec?

UNIVERSITAT
ROVIRA i VIRGILI

